

Q-Kernel Note - fallback text render

```
\documentclass[11pt]{article}

\usepackage[margin=1in]{geometry}
\usepackage{amsmath,amssymb,amsthm}
\usepackage{hyperref}
\usepackage{enumitem}
\usepackage{xcolor}

\title{Q-Kernel: Exact Odd- $(Q)$  Contextuality Kernel Extraction for Weyl/Pauli Measurement Programs}
\author{Manuel Flores Gordillo}
\date{\today}

\newtheorem{definition}{Definition}
\newtheorem{proposition}{Proposition}
\newtheorem{remark}{Remark}

\begin{document}
\maketitle

\begin{abstract}
Q-Kernel is a research-software implementation of an exact odd- $(Q)$  contextuality criterion for finite Weyl/Pauli measurement programs. Given an explicitly specified context family, the library builds a context-observable incidence matrix over  $(\mathbb{F}_2)$ , computes integer commutator-carry bits, searches for a closed cycle with odd carry, extracts a smaller contextuality kernel, and exports independently verifiable certificates. The implementation is deliberately conservative: it is a contextuality-kernel extractor, not a magic-state optimizer or a hardware-resource estimator. The package provides multiple exact solvers, connected-component decomposition, metadata-aware observable identity semantics, SAT/MaxSAT export, external-solver output import, and stable certificate verification.
\end{abstract}

\section{Purpose and scope}
```

The goal of Q-Kernel is to turn an exact finite contextuality criterion into a reproducible compiler-analysis primitive. The novelty target is the odd- (Q) carry/kernel/certificate implementation, not the $GF(2)$ linear-system framing itself. The input is not a raw list of Pauli operators, but a measurement program: observables together with the contexts in which they are jointly measured or algebraically asserted.

The safe claim is:

```
[
\text{Q-Kernel extracts exact odd-}Q\text{ contextuality kernels.}
]
```

The unsafe claim is:

```
[
\text{Q-Kernel is a magic-state, }T\text{-count, or distillation optimizer.}
]
```

The latter would require an additional bridge theorem or robust empirical evidence. This paper therefore presents Q-Kernel as a certificate-producing contextuality analyzer.

```
\section{Related work and novelty hygiene}
```

The binary multi-qubit case has an established finite-geometric formulation in terms of symplectic polar spaces over $(\mathbb{F}_2)$. In particular, de Boutray, Holweck, Giorgetti, Masson, and Saniga formulate a broad class of observable-based contextuality proofs as the absence of solutions to a linear system and study contextuality degree for subgeometries of binary symplectic polar spaces~\cite{deBoutrayHolweckGiorgettiMassonSaniga2022}. Subsequent work by Muller, Saniga, Giorgetti, de Boutray, and Holweck develops algorithms and bounds for contextuality degree in multi-qubit symplectic polar spaces~\cite{MullerSanigaGiorgettiDeBoutrayHolweck2023}.

Q-Kernel therefore does not claim novelty for the linear-system framing, the use of $(\mathbb{F}_2)$ incidence constraints, or the binary symplectic polar-space viewpoint. Its contribution is narrower: an exact software

implementation of the odd- $\mathbb{Q}$ carry criterion with integer-lift arithmetic, a closed symplectic $\mathbb{Q}$ -form, qudit-aware scope beyond purely binary polar-space inputs, minimum-kernel extraction, explicit observable/event identity semantics, external SAT/MaxSAT export/import, and standalone verifiable certificates.

`\section{Weyl/Pauli measurement programs}`

Fix local dimension $\mathbb{Q}(d)$ and $\mathbb{Q}(m)$ sites. Q-Kernel represents Weyl labels as vectors

$$v \in \mathbb{Z}_{d^2}^{2m}$$

with interleaved coordinates

$$[z_1, x_1, z_2, x_2, \dots, z_m, x_m].$$

The integer symplectic pairing is

$$\langle v, w \rangle = \sum_{k=1}^m (x_k z'_k - z_k x'_k),$$

where the right-hand side is evaluated on integer lifts.

A context $\mathbb{C}=(c_1, \dots, c_r)$ is valid when the labels commute pairwise and sum to zero modulo $\mathbb{Q}(d)$. The commutator carry bit is

$$b(\mathbb{C}) = \sum_{i < j} \frac{\langle c_i, c_j \rangle}{d} \pmod{2}.$$

The division by $\mathbb{Q}(d)$ is performed before reducing modulo $\mathbb{Q}(2)$. Reducing the symplectic pairing modulo $\mathbb{Q}(d)$ first destroys the carry information and gives the wrong invariant.

`\begin{proposition}[Integer carry rule]`
`\label{prop:integer-carry}`
 For a valid context $\mathbb{C}$, pairwise commutation implies $\langle c_i, c_j \rangle \equiv 0 \pmod{d}$, so each quotient $\langle c_i, c_j \rangle / d$ is defined on integer lifts. The carry bit $b(\mathbb{C})$ is the parity of the sum of these integer quotients. It is not recoverable from the symplectic pairings after reducing them modulo $\mathbb{Q}(d)$.
`\end{proposition}`

In the implementation this rule is enforced by `\texttt{qkernel.symplectic.symplectic\_int}` and `\texttt{qkernel.carry.compute\_b}`.

`\section{Incidence cycles and the odd- $\mathbb{Q}$  criterion}`

Given contexts $\mathbb{C}_1, \dots, \mathbb{C}_n$ and observable names $\mathbb{O}_1, \dots, \mathbb{O}_s$, Q-Kernel builds the incidence matrix

$$A \in \mathbb{F}_2^{n \times s}$$

where $A_{ij}=1$ iff observable $\mathbb{O}_j$ appears in context $\mathbb{C}_i$.

A binary vector $\lambda \in \mathbb{F}_2^n$ selects a collection of contexts. It is closed when

$$A^T \lambda = 0.$$

For even (d) , the implemented criterion is

$$\exists \lambda. A^T \lambda = 0, b^T \lambda = 1.$$

Implemented odd- (Q) criterion

For the even- (d) Weyl context families covered by the underlying criterion note, contextuality is witnessed exactly by a binary vector (λ) satisfying $(A^T \lambda = 0)$ and $(b^T \lambda = 1)$. Q-Kernel implements this condition as its certified contextuality test.

The quantity $(b^T \lambda)$ is the odd- (Q) obstruction.

Closed symplectic form for (Q)

Closed symplectic form for (Q)

For a closed cycle (λ) , flatten the selected contexts into an ordered observable multiset $(v_1, \dots, v_N)$. Then

$Q(\lambda)$

$=$

$\frac{1}{d} \sum_{a < b}$

$\langle v_a, v_b \rangle$

$\pmod{2}$.

The division by (d) is applied to the aggregate integer numerator. Individual inter-context pairings need not be divisible by (d) .

The division by (d) is applied to the aggregate integer numerator. Individual inter-context pairings need not be divisible by (d) .

The closed form agrees with $(b^T \lambda)$. Each selected context sum can be written in an integer lift as $(S_r = d t_r)$. Inter-context terms contribute

$\frac{1}{d} \langle S_r, S_s \rangle$

$=$

$d \langle t_r, t_s \rangle$,

which vanishes modulo (2) for even (d) . Thus the aggregate symplectic numerator reduces modulo (2) to the sum of intra-context carry terms.

The implementation exposes both computations in `qkernel.closed_form` and cross-checks them on cycle-space elements.

The implementation exposes both computations in `qkernel.closed_form` and cross-checks them on cycle-space elements.

The implementation exposes both computations in `qkernel.closed_form` and cross-checks them on cycle-space elements.

Kernelization problem

Q-Kernel solves the minimum-context odd- (Q) kernel problem:

$\min_{\lambda} \quad$

$\text{s.t.} \quad A^T \lambda = 0, \quad b^T \lambda = 1, \quad \lambda \in \{0, 1\}^n.$

The extracted kernel is the selected subprogram induced by (λ) . Q-Kernel also reports the selected observable set and compression ratios, but those ratios are diagnostic; they are not resource monotones.

Observable identity semantics

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

A central implementation issue is the distinction between algebraic equality and protocol identity. Two labels may have the same Weyl vector while representing different measurement events in a circuit or schedule.

Q-Kernel therefore attaches optional metadata to observables:

```
\[
\texttt{identity\_scope}\in\{\texttt{observable},\texttt{event}\}.
\]
```

Safe canonicalization by Weyl vector merges observable-scoped labels only. Event-scoped labels remain distinct unless an explicit force mode is selected. This avoids silently creating parity cycles that are algebraically valid but protocol-false.

```
\section{Input frontends}
```

The package currently supports four input styles:

```
\begin{enumerate}[leftmargin=*]
\item Weyl JSON programs.
\item Pauli context JSON programs.
\item Grouped Pauli schedules.
\item Row-oriented Pauli table CSV/JSON.
\end{enumerate}
```

The Pauli table adapter is a dependency-free contract intended for future Qiskit, Stim, or compiler-specific frontends. A raw set of observables is not enough; contexts or layers must be explicit.

```
\section{Algorithms and solvers}
```

Q-Kernel provides several exact pure-Python solvers:

```
\begin{description}[leftmargin=*]
\item[Cycle-span enumeration.] Enumerates the span of a basis for  $\ker A^T$ . Exact and simple, but exponential in cycle-space dimension.
\item[Bounded-weight search.] Searches subsets by increasing Hamming weight up to a user budget. Useful when small certificates are expected.
\item[Branch-and-bound.] Encodes each context as
\[
\texttt{state}_i = \texttt{incidence\_row}_i \text{Vert} b_i
\]
```

and searches for XOR target $0\dots 01$, pruning any branch whose remaining suffix span cannot reach the residual target.

```
\end{description}
```

The library also decomposes disconnected context--observable components. If a global odd- (Q) cycle exists in a disconnected union, at least one connected component contains an odd- (Q) cycle. This allows independent component search and prevents irrelevant disconnected blocks from inflating the kernel search.

```
\begin{proposition}[Component decomposition]
\label{prop:component-decomposition}
Let a context family decompose into disconnected components in the context--observable bipartite graph. If a global cycle has odd  $(Q)$ , then at least one component has an odd- $(Q)$  cycle. Consequently, the minimum-cardinality odd- $(Q)$  kernel of the disconnected union is the minimum among the componentwise odd- $(Q)$  kernels.
\end{proposition}

\begin{proof}
Both incidence closure and the carry parity decompose as direct sums over disconnected components. A global closed vector  $(\lambda)$  is the sum of its component restrictions, and  $(Q(\lambda))$  is the parity sum of the component  $(Q)$ -values. If the total parity is odd, at least one component parity is odd. The weight of that component restriction is no larger than the global weight. Conversely, any odd component cycle is an odd global cycle after extending by zero.
\end{proof}

\section{External SAT and MaxSAT workflows}
```

The minimum odd- (Q) problem is naturally a Boolean optimization problem. Q-Kernel exports dependency-free solver models:

```
\begin{itemize}[leftmargin=*]
```

\item DIMACS CNF for fixed- k feasibility:

$$A^T \lambda = 0, \quad b^T \lambda = 1, \quad |\lambda| \leq k.$$

\item WCNF/MaxSAT for direct minimization, with parity constraints as hard clauses and unit soft clauses
 λ_i penalizing selected contexts.

\end{itemize}

External solvers are treated as untrusted candidate generators. Q-Kernel imports only the selected λ variables from solver output and independently verifies the resulting certificate.

\section{Certificates}

A Q-Kernel certificate records:

\begin{itemize}[leftmargin=*]

\item a canonical SHA-256 hash of the input program;

\item the selected vector λ ;

\item selected context and observable indices;

\item the carry vector b ;

\item the claimed Q -value;

\item software version, coordinate convention, integer-carry rule, and criterion identifier;

\item the independent verifier result.

\end{itemize}

\begin{proposition}[Certificate verification soundness]

\label{prop:certificate-soundness}

If the Q-Kernel certificate verifier accepts a certificate for a supplied program, then the certificate is bound to that program hash, its selected contexts define the claimed vector λ , the vector is closed under the program incidence matrix, and the recomputed odd- Q value is 1.

\end{proposition}

Verification recomputes the program hash, checks λ against selected contexts, verifies $A^T \lambda = 0$, and recomputes $Q(\lambda) = 1$.

\section{Peres--Mermin test case}

For the two-qubit Peres--Mermin square, Q-Kernel recovers the standard six-context odd- Q kernel. The carry vector has a single odd entry in the implemented convention, and the selected cycle is

$$\lambda = (1, 1, 1, 1, 1, 1).$$

The analyzer reports

$$Q(\lambda) = 1.$$

Negative controls, including row-only subfamilies and event-scoped duplicate labels, are noncontextual unless explicit force-canonicalization is requested.

\section{Synthetic benchmarks}

The current benchmark suite is intentionally modest. It measures software behavior rather than quantum resource value. Examples include:

\begin{itemize}[leftmargin=*]

\item Peres--Mermin plus disconnected noncontextual noise blocks;

\item connected zero-carry ladders;

\item solver comparison across span, bounded-weight, branch-and-bound, SAT export, and MaxSAT export workflows.

\end{itemize}

These benchmarks show that decomposition and solver choice matter, but they do not establish hardware

advantage or magic-resource bounds.

```
\input{generated_benchmark_tables}
\section{Tower and doubling scope}
```

The tower/doubling direction is deliberately experimental. Current evidence supports a repeat-free subclass where lift-induced carry has a closed bit formula. Repeat-observable corrections remain open. Q-Kernel therefore exposes only a scope reporter for tower/doubling experiments and does not certify tower compression, free resource activation, or an additive resource-gauge interpretation.

```
\section{Limitations}
```

Q-Kernel does not currently claim:

```
\begin{itemize}[leftmargin=*]
\item a magic-state monotone;
\item a  $\backslash(T)$ -count or  $\backslash(T)$ -depth lower bound;
\item stabilizer-rank estimates;
\item hardware overhead estimates;
\item certified tower/doubling compression;
\item that passive dimension embedding is operationally free.
\end{itemize}
```

The odd- $\backslash(Q)$ obstruction is an exact contextuality certificate. Any resource-estimation interpretation is future work.

```
\section{Roadmap}
```

Immediate technical directions:

```
\begin{enumerate}[leftmargin=*]
\item Optional PySAT/RC2 integration while keeping the core dependency-light.
\item Stim-lite measurement-event table adapter.
\item Optional Qiskit bridge emitting the Pauli table format.
\item More realistic benchmark families.
\item Empirical correlation studies between kernel metrics and known circuit-resource metrics, without treating correlation as theorem.
\item Further formalization of tower/doubling laws, especially repeat-observable corrections.
\end{enumerate}
```

```
\section{Code-to-proposition map}
```

The main theorem-level claims are intentionally tied to implementation modules:

```
\begin{itemize}[leftmargin=*]
\item Proposition~\ref{prop:integer-carry}: \texttt{symplectic.py}, \texttt{carry.py},
\texttt{test\_integer\_carry\_bug.py}.
\item Proposition~\ref{prop:odd-q-criterion}: \texttt{incidence.py}, \texttt{analyzer.py}, \texttt{optimizer.py}.
\item Proposition~\ref{prop:closed-q-form}: \texttt{closed\_form.py}, \texttt{test\_closed\_q\_form.py}.
\item Proposition~\ref{prop:component-decomposition}: \texttt{decompose.py}, \texttt{test\_decompose.py}.
\item Proposition~\ref{prop:certificate-soundness}: \texttt{certificate.py}, \texttt{verify.py},
\texttt{solver\_output.py}.
\end{itemize}
```

```
\bibliographystyle{plain}
\bibliography{references}

\end{document}
```